ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 3

INTRODUCTION TO TECHNICAL PROGRAMMING

PAPER 1 · PRACTICE D · MARKING GUIDELINE

MARKS: 120

Marking guideline for the Academic Central NC(V) Level 3 Introduction to Technical Programming paper 1 · practice d.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 3. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Code is not marked strictly on syntax: give marks for correct logic and structure. Accept other correct answers, other variable names and other correct flowchart layouts.

SECTION A

QUESTION 1

1.1 D ✓
1.2 B ✓
1.3 A ✓
1.4 C ✓
1.5 B ✓
1.6 A ✓
1.7 D ✓
1.8 C ✓
1.9 A ✓
1.10 B ✓

(10)

[10]

QUESTION 2

2.1 True ✓
2.2 False ✓
2.3 True ✓
2.4 False ✓
2.5 True ✓

(5)

[5]

QUESTION 3

3.1 C ✓
3.2 B ✓
3.3 E ✓
3.4 G ✓
3.5 I ✓
3.6 F ✓
3.7 J ✓
3.8 A ✓
3.9 H ✓
3.10 D ✓

(10)

[10]

QUESTION 4

4.1 Preprocessor directive (includes the iostream library) ✓

4.2 Namespace statement ✓

4.3 main function header (start of the program) ✓

4.4 Variable declaration and initialisation ✓

4.5 Output statement ✓

4.6 Return statement (ends main and returns 0 to the operating system) ✓

(6)

[6]

TOTAL SECTION A: [31]

SECTION B

QUESTION 5

5.1.1 An error in the logic of a program ✓: the program runs but gives the wrong result (e.g. using + instead of −) ✓. (2)

5.1.2 A named value that cannot be changed while the program runs ✓✓. Example ✓✓: const double VAT = 0.15; (4)

5.1.3 Motherboard ✓✓ (2)

5.1.4 Graphics card / graphics processing unit (GPU) ✓✓ (2)

5.1.5 Finding ✓ and fixing (removing) errors in a program ✓. (2)

5.2 Repeating a set of instructions ✓ a number of times or until a condition is met (a loop) ✓. (2)

5.3 Decomposition ✓; pattern recognition ✓; abstraction ✓; algorithm design ✓ (4)

5.4 A value-returning function sends a result back to the code that called it with a return statement ✓, and has a return type such as int or double ✓ (e.g. int add(int a, int b)). A void function does a task but returns no value ✓, and its type is void ✓ (e.g. void showMenu()). (4)

5.5 Any TWO, ✓✓ each: no syntax errors because blocks only fit together in valid ways; easy to see the program's structure; quick results with sprites and sounds keep beginners interested; focuses on logic instead of typing. (4)

[26]

QUESTION 6

Understand the problem ✓: identify the inputs (amount due, amount paid), output (change and the number of R10, R5 and R1) and conditions (paid must be at least the amount due) ✓✓. Devise a plan ✓: change = paid − due; use integer division and the remainder (% operator) to find the notes and coins; write pseudocode / a flowchart ✓✓. Carry out the plan ✓: write the C++ code, using cin, calculations and cout ✓✓. Look back ✓: test with known values (e.g. due R27, paid R50 → R23 = 2 × R10, 0 × R5, 3 × R1), fix errors and check whether the program can be improved ✓✓. (11)

[11]

QUESTION 7

7.1 number – the times table chosen by the user ✓; count – the number to multiply by (1 to 10) ✓ (2)

7.2 10 sums ✓; the "repeat 10" block ✓ (2)

7.3 It checks that the number entered is from 1 to 12 ✓ and stops the program with a message if it is not (input validation) ✓. (2)

7.4 7 x 1 = 7 ✓✓ (2)

7.5 5 x 10 = 50 ✓✓ (2)

7.6 Libraries and main() ✓; variables ✓; prompt and cin ✓; if with || for the range check ✓ and a message ✓; for loop from 1 to 10 ✓; correct output of each sum ✓; return 0 / structure ✓, e.g. (8)

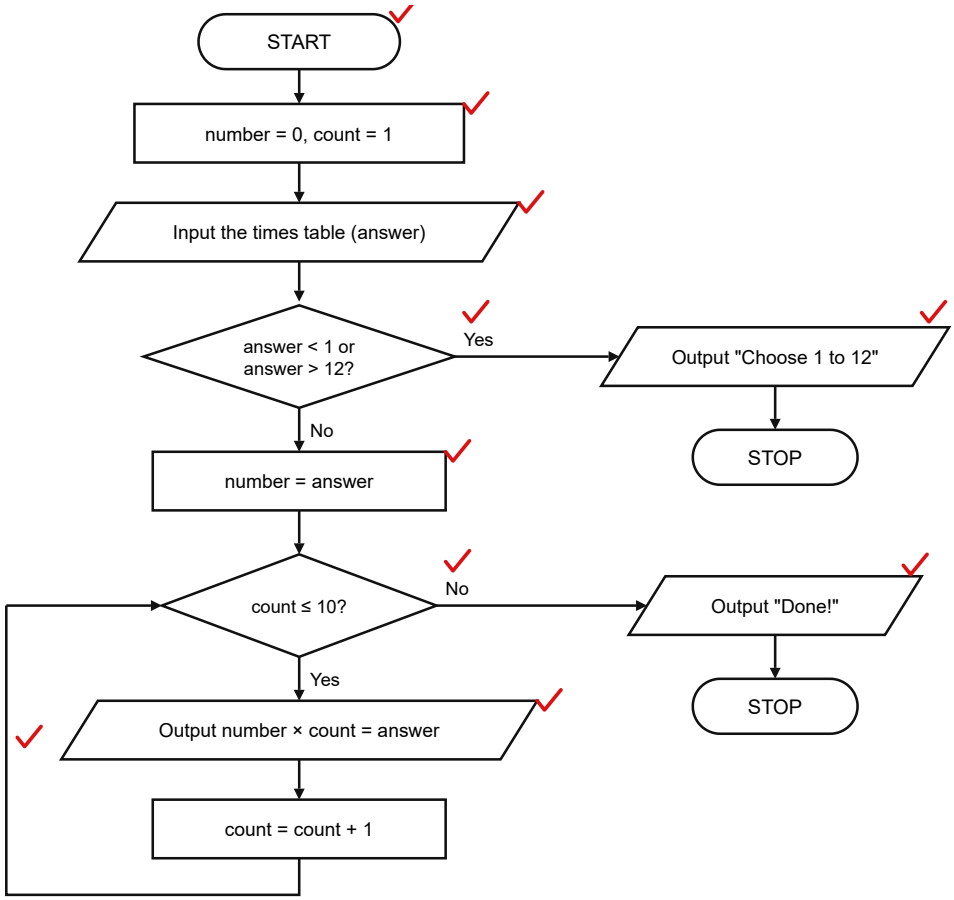
```
#include <iostream>
using namespace std;

int main() {
    int number;
    cout << "Which times table do you want to practise (1 to 12)? ";
    cin >> number;
    if (number < 1 || number > 12) {
        cout << "Choose a number from 1 to 12" << endl;
    } else {
        for (int count = 1; count <= 10; count++) {
            cout << number << " x " << count << " = " << number * count << endl;
        }
        cout << "Done!" << endl;
    }
    return 0;
}
```

7.7 The first sum must be number × 1 ✓; starting at 0 would show a "× 0" sum and stop at × 9 ✓. (2)

7.8 The "repeat 10" block ✓ must be changed to "repeat 12" ✓. (2)

7.9



(Start 1; number = 0 and count = 1 1; input the times table 1; decision answer < 1 or > 12? 1; Yes → message and stop 1; No → number = answer 1; loop decision count ≤ 10? 1; output the sum 1; count = count + 1 and loop back 1; output "Done!" and stop 1)

(10)

[32]

QUESTION 8

8.1 A do … while loop ✓ that repeats forever (while true) until break is reached when 0 is entered ✓. (2)

8.2 It checks the value of choice ✓ and adds the price of the chosen item × quantity to the bill ✓. (2)

8.3 It leaves the switch after the matching case ✓, so the code of the next cases is not also run ✓. (2)

8.4 When choice is not 1, 2 or 3 ✓ (and not 0) – a number that is not on the menu ✓. (2)

8.5 $2 \times 18 + 1 \times 12$ ✓ $= \text{R}48\ (\text{no discount})$ ✓ (2)

8.6 $4 \times 18 + 3 \times 15 = 117; 117 \times 0{,}9$ ✓ $= \text{R}105{,}30$ ✓ (2)

8.7 It adds the value on the right to the variable ✓ and stores the result in the same variable (bill = bill + 18 * qty) ✓. (2)

8.8 So that the program goes straight to paying when the cashier enters 0 ✓, without asking for a quantity that is not needed ✓. (2)

8.9 double ✓ – the bill can have cents after the discount (e.g. 105.30) ✓. (2)

8.10 Check that the choice is 1, 2 or 3 before asking for the quantity ✓ (e.g. an if statement or a loop that asks again until the choice is valid), and show "Not on the menu" otherwise ✓. (2)

[20]

TOTAL SECTION B: [89]

TOTAL: 120

END OF MARKING GUIDELINE